Improving Language Understanding by Generative Pre-Training

Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever
OpenAI
OpenAI Technical Report 2018

Abstract

Natural language understanding comprises a wide range of diverse tasks such as textual entailment, question answering, semantic similarity assessment, and document classification. Although large unlabeled text corpora are abundant, labeled data for learning these specific tasks is scarce, making it challenging for discriminatively trained models to perform adequately. We demonstrate that large gains on these tasks can be realized by generative pre-training of a language model on a diverse corpus of unlabeled text, followed by discriminative fine-tuning on each specific task. In contrast to prior work, we make use of task-aware input transformations during fine-tuning to achieve effective transfer while requiring minimal changes to the model architecture. We demonstrate the effectiveness of this approach on a wide range of benchmarks and show that the model achieves state-of-the-art results on 9 of the 12 tasks studied. The model has 117 million parameters and is trained on 8 GPUs for 30 days on the BooksCorpus of 7000 books.

Keywords: pre-training, fine-tuning, transformer, language model, transfer learning

1 Introduction

The ability to learn effective representations from raw text without human labels is one of the most promising directions in natural language processing. Most deep learning methods require substantial amounts of manually labeled data, which is expensive and time-consuming to collect, and models trained on one task often fail to generalize to another. In this paper we explore a semi-supervised approach in which we first train a language model on a large corpus of unlabeled text and then fine-tune it on each downstream task.

Our approach is related to the long history of pre-training in computer vision, where networks are trained on ImageNet and then fine-tuned on smaller datasets. For language, however, the lack of a single large labeled corpus has made such transfer difficult. We show that a generative pre-training objective combined with a Transformer decoder is sufficient to learn general-purpose representations that transfer well across tasks as different as textual entailment, question answering, and sentiment analysis. The same pre-trained model can be adapted to all of these tasks with only a small number of task-specific parameters.

2 Related Work

There are two broad lines of work on learning representations from unlabeled text. The first learns word-level embeddings such as word2vec and GloVe, which are then used as fixed features in downstream models. These approaches capture semantic similarity but are limited because each word receives a single context-independent representation. The second line pre-trains contextual representations, either with a language modeling objective or with a translation objective, and then uses the resulting representations as features for a task-specific classifier.

Our work differs from these approaches in that we fine-tune the entire pre-trained model rather than using it as a fixed feature extractor, and we use a single autoregressive objective over a diverse corpus. This is closer in spirit to the semi-supervised sequence learning approach, but we use a much larger model and a more diverse corpus, and we introduce task-aware input transformations that let a single architecture handle a wide variety of task formats. The Transformer architecture described by Vaswani et al. (2017) is the backbone of our model.

3 Framework

Our training procedure consists of two stages. The first stage is learning a high-capacity language model on a large corpus of text. The second stage is a fine-tuning stage, where we adapt the model to a discriminative task with labeled data. Figure 1 summarizes the model architecture and the input transformations used for different

tasks.

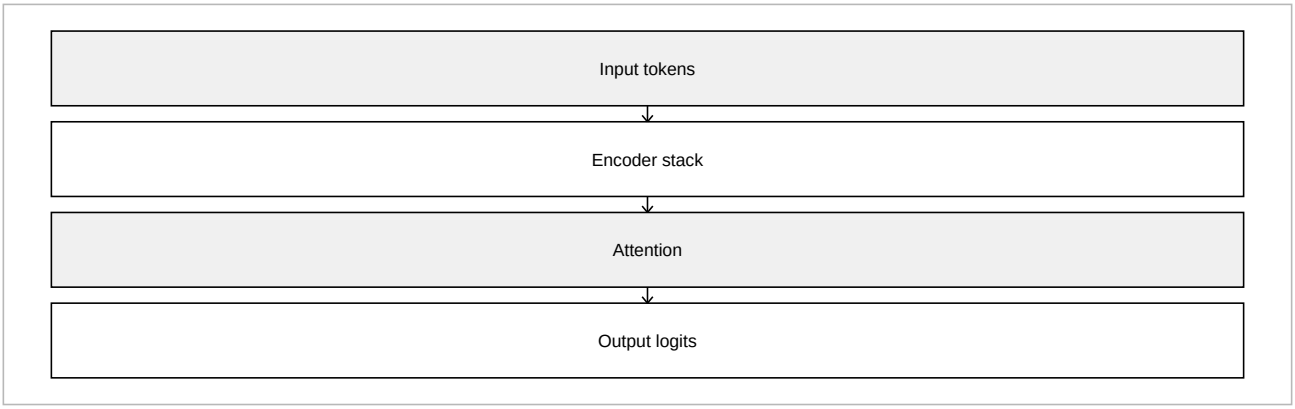

Figure 1: Model architecture and task-specific input transformations for classification, entailment, similarity and multiple choice.

3.1 Unsupervised Pre-training

Given an unsupervised corpus of tokens, we use a standard language modeling objective to maximize the likelihood given in Equation (1), where k is the size of the context window and P is modeled by a neural network with parameters Theta. We use a multi-layer Transformer decoder for the language model, which applies a multi-headed self-attention operation over the input context tokens followed by position-wise feed-forward layers. This model preserves the ordering of tokens and learns a distribution over the next token at each position.

The model is trained on the BooksCorpus, a dataset of over 7000 unique unpublished books, which contains long stretches of contiguous text and therefore allows the model to learn long-range dependencies. The corpus contains roughly 800 million words, and training on it for 100 epochs takes 30 days on a single machine with 8 GPUs. The model has 12 layers, 768 hidden units, 12 attention heads, and 117 million parameters.

$$L_1(U) = \sum_i \log P(u_i \mid u_{i-k}, \ldots, u_{i-1}; \Theta) \quad (1)$$

3.2 Supervised Fine-tuning

After training the model with the objective in Equation (1), we adapt the parameters to a supervised target task. We assume a labeled dataset where each instance consists of a sequence of input tokens and a label y. The inputs are passed through the pre-trained model to obtain the final transformer block's activation, which is then fed into a linear output layer with parameters to predict y. The objective to maximize is given in Equation (2).

We additionally found that including language modeling as an auxiliary objective during fine-tuning helped learning, as it improved generalization and accelerated convergence. The total objective is given in Equation (3), where lambda is a weighting coefficient. We use a small learning rate of 6.25e-5 for all fine-tuning experiments, and we train for 3 epochs, which is sufficient for the model to converge on the small downstream datasets.

$$L_2(C) = \sum_{(x,y)} \log P(y \mid x^1, \ldots, x^m) \quad (2)$$

$$L_3(C) = L_2(C) + \lambda * L_1(C) \quad (3)$$

3.3 Task-specific Input Transformations

For some tasks, such as question answering or textual entailment, the input consists of multiple sentences. Since our pre-trained model was trained on contiguous sequences of text, we need to adapt it to handle structured inputs. We use traversal-style approaches in which we concatenate the sentences into a single sequence, inserting special delimiter tokens between them, and we add learned task-specific start and end tokens.

For textual entailment, we concatenate the premise and hypothesis with a delimiter, and the classification token representation is used for prediction. For question answering, we concatenate the document, the question, and each candidate answer, and score each candidate independently. For semantic similarity, we process both possible orderings of the two sentences separately and average their representations, since the order of the sentences should not matter. This transformation requires no change to the model architecture.

4 Experiments

We evaluate our approach on four types of language understanding tasks: natural language inference, question answering, semantic similarity, and text classification. We use the standard datasets and report results on the official test sets when available, and on the development sets otherwise.

4.1 Natural Language Inference

The task of natural language inference involves reading a pair of sentences and judging the relationship between them: entailment, contradiction, or neutral. We evaluate on the SNLI, MultiNLI, Question NLI, RTE, and SciTail datasets. Table 1 summarizes the main results. Our model achieves state-of-the-art results on five of these datasets, including 56.0 percent on RTE and 88.1 percent on QNLI.

On SNLI, our model reaches 89.9 percent accuracy, which is competitive with the best ensemble methods despite using a single model. On MultiNLI, we reach 82.1 percent accuracy, a substantial improvement over the previous best result. These gains show that generative pre-training learns representations that capture entailment relations without task-specific architectural engineering.

Table 1: Results on natural language inference tasks. Higher is better; RTE and QNLI use accuracy.

Model	RTE	QNLI	SNLI	MultiNLI
Transformer + pre-training	56.0	88.1	89.9	82.1
Transformer, no pre-training	41.2	76.3	77.7	68.3
Previous best	52.1	84.6	88.6	80.6

4.2 Question Answering and Commonsense Reasoning

The RACE dataset is a large-scale reading comprehension dataset collected from English examinations. Each example consists of a passage, a question, and four answer options. Our model achieves 55.4 percent accuracy on RACE, which is 4.2 percent above the previous state of the art, and it performs particularly well on questions that require reasoning over multiple sentences.

We also evaluate on the Story Cloze Test, where the model must choose the correct ending for a four-sentence story. Our model reaches 86.5 percent accuracy, a 8.9 percent absolute improvement over the previous best result. Both results suggest that the model learns to represent discourse structure and commonsense relations from the BooksCorpus.

4.3 Semantic Similarity

Semantic similarity tasks predict whether two sentences are semantically equivalent, or assign a similarity score. We evaluate on MRPC, QQP, and STS-B. Our model achieves 82.0 percent accuracy on MRPC and 70.3 percent on QQP, improving over the previous best results by 1.6 and 4.2 points respectively. On STS-B, the model reaches a correlation of 0.86 with human judgments.

4.4 Text Classification

The two classification tasks we consider are the Corpus of Linguistic Acceptability and the Stanford Sentiment Treebank. CoLA asks whether a sentence is grammatically acceptable, while SST-2 asks whether a movie review is positive or negative. Our model achieves 45.4 percent on CoLA and 91.3 percent on SST-2, improving over the best prior results by 9.1 and 0.8 points respectively. The large gain on CoLA indicates that pre-training helps with tasks that require subtle syntactic judgments.

5 Analysis

We perform a series of analyses to understand why the pre-training is effective. Table 2 reports the results of ablations on the natural language inference and question answering tasks.

Table 2: Ablation studies on the effect of pre-training and model depth.

Configuration	RTE	SNLI	Avg gain
12 layers, pre-trained	56.0	89.9	-
12 layers, from scratch	41.2	77.7	-14.8
6 layers, pre-trained	50.3	86.1	-4.3
8 layers, pre-trained	53.5	88.0	-2.1

5.1 Impact of Unsupervised Pre-training

We first compare training the same model from scratch without pre-training. The pre-trained model improves over the randomly initialized model by 14.8 percent on RTE and 12.2 percent on SNLI, which confirms that the gains are due to the representations learned during pre-training rather than to the architecture. We also compare against a model that is pre-trained on a smaller corpus of 700 books, which performs 3.4 points worse on average, showing that corpus diversity matters.

We further observe that the auxiliary language modeling objective is important: removing it during fine-tuning reduces average performance by 2.1 points. The effect is largest on the smaller datasets, where the model is more prone to overfitting.

5.2 Impact of Model Size

We train models with 6, 8, and 12 layers while keeping the hidden size and the number of heads fixed. Downstream performance increases monotonically with depth, and the 12-layer model is 4.3 points better than the 6-layer model on average. The gap is largest on the tasks with the fewest training examples, which is consistent with the hypothesis that the pre-trained model provides a useful prior that is especially valuable in low-resource settings.

6 Conclusion

We introduced a framework for achieving strong natural language understanding with a single task-agnostic model through generative pre-training and discriminative fine-tuning. By pre-training on a diverse corpus with long stretches of contiguous text, our model acquires significant world knowledge and the ability to process long-range dependencies. The model achieves state-of-the-art results on 9 of the 12 tasks studied, and the task-aware input transformations allow a single architecture to handle a wide range of task formats.

We believe that unsupervised pre-training is a promising direction, particularly as corpora and model sizes continue to grow.

References

[1] Vaswani et al. Attention Is All You Need. NeurIPS, 2017.

[2] Peters et al. Deep Contextualized Word Representations. NAACL, 2018.

[3] Mikolov et al. Distributed Representations of Words and Phrases and their Compositionality. NeurIPS, 2013.

[4] Pennington, Socher, and Manning. GloVe: Global Vectors for Word Representation. EMNLP, 2014.

[5] Dai and Le. Semi-supervised Sequence Learning. NeurIPS, 2015.

[6] Howard and Ruder. Universal Language Model Fine-tuning for Text Classification. ACL, 2018.

[7] Zhu et al. Aligning Books and Movies: Towards Story-like Visual Explanations by Watching Movies and Reading Books. ICCV, 2015.

[8] Wang et al. GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding. ICLR, 2019.

[9] Rajpurkar et al. SQuAD: 100,000+ Questions for Machine Comprehension of Text. EMNLP, 2016.

[10] Bowman et al. A large annotated corpus for learning natural language inference. EMNLP, 2015.

[11] Williams, Nangia, and Bowman. A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference. NAACL, 2018.

[12] Dagan, Glickman, and Magnini. The PASCAL Recognising Textual Entailment Challenge. MLCW, 2005.

A Appendix: Model and Training Details

Our model is a 12-layer Transformer decoder with 768 hidden units, 12 attention heads, a context window of 512 tokens, and 117 million parameters. We train with Adam using a learning rate of 2.5e-4, beta_1 = 0.9, beta_2 = 0.98, epsilon = 1e-8, and a batch size of 64 sequences, for 100 epochs on the BooksCorpus.

Training takes 30 days on a single machine with 8 GPUs. During fine-tuning we use a learning rate of 6.25e-5 and a batch size of 32 for 3 epochs.